Assignment 19.1

R. Abhinav Reddy

2303A52348

Batch: 44

Task 1: Python to Java Conversion

```python
def task1_python_to_java(work_dir: Path) -> TaskResult:
    print("\n# Task 1 - Python to Java Conversion")

    python_class_code = textwrap.dedent(
        """
        class Student:
            def __init__(self, name, marks):
                self.name = name
                self.marks = marks

            def is_pass(self):
                return self.marks >= 40

            def display(self):
                status = "PASS" if self.is_pass() else "FAIL"
                return f"Student: {self.name}, Marks: {self.marks}, Result: {status}"
        """
    ).strip()

    java_class_code = textwrap.dedent().strip()

    print("Python class source:")
    print(python_class_code)
    print("\nTranslated Java class:")
    print(java_class_code)

    class Student:
        def __init__(self, name: str, marks: int) -> None:
            self.name = name
            self.marks = marks

        def is_pass(self) -> bool:
            return self.marks >= 40

        def display(self) -> str:
            status = "PASS" if self.is_pass() else "FAIL"
            return f"Student: {self.name}, Marks: {self.marks}, Result: {status}"

    py_output = Student("Asha", 72).display()
    print("\nPython output:")
    print(py_output)

    expected_output = "Student: Asha, Marks: 72, Result: PASS"

    java_file = work_dir / "Student.java"
    java_file.write_text(java_class_code + "\n", encoding="utf-8")
```

Output:

```
Student: Asha, Marks: 72, Result: PASS
```

Observation:

1. Task 1 - Python to Java Conversion
   The translated Java class preserves class structure, attributes, constructor, and method behaviour from Python. Output logic is equivalent, and expected result is correct. Full Java runtime verification is pending on your machine due to missing Java installation.

Task 2: C++ to Python Function Conversion

Code:

```python
def task2_cpp_to_python_factorial() -> TaskResult:
    print("\n# Task 2 - C++ to Python Function Conversion")

    cpp_code = textwrap.dedent(
        """
        #include <iostream>
        using namespace std;

        long long factorial(int n) {
            if (n <= 1) return 1;
            return n * factorial(n - 1);
        }

        int main() {
            cout << factorial(5) << endl;
            return 0;
        }
        """
    ).strip()

    py_code = textwrap.dedent(
        """
        def factorial(n: int) -> int:
            if n < 0:
                raise ValueError("n must be non-negative")
            if n <= 1:
                return 1
            return n * factorial(n - 1)
        """
    ).strip()

    print("Original C++ function:")
    print(cpp_code)
    print("\nTranslated Python function:")
    print(py_code)
```

Output:

```
Validation outputs:
factorial(0) = 1
factorial(1) = 1
factorial(5) = 120
factorial(7) = 5040
Observation: Recursive translation is correct and follows Pythonic style with type hints and input
guard.
```

Observation:

The recursive factorial translation is functionally correct and follows Python conventions (type hints, clear base case, input validation for negatives). Sample tests confirm correct outputs.

Task 3: SQL to Pandas Query

Code:

```python
def task3_sql_to_pandas() -> TaskResult:
    print("\n# Task 3 - SQL to Pandas Query")

    sql_query = "SELECT name, salary FROM employees WHERE salary > 50000;"
    pandas_equivalent = "result = employees.loc[employees['salary'] > 50000, ['name', 'salary']]"

    employees = pd.DataFrame(
        {
            "name": ["Asha", "Ravi", "Neha", "Kiran"],
            "salary": [48000, 62000, 51000, 45000],
            "dept": ["HR", "IT", "Finance", "Sales"],
        }
    )
    result = employees.loc[employees["salary"] > 50000, ["name", "salary"]]

    print("SQL query:")
    print(sql_query)
    print("\nEquivalent Pandas:")
    print(pandas_equivalent)
    print("\nResult DataFrame:")
    print(result.to_string(index=False))

    expected_names = ["Ravi", "Neha"]
    passed = result["name"].tolist() == expected_names
    observation = "Pandas filter and column projection exactly match SQL WHERE + SELECT behavior."
    print("Observation:", observation)
    return TaskResult(task="Task 3", observation=observation, passed=passed)
```

Output:

```
Result DataFrame:
name  salary
Ravi   62000
Neha   51000
```

Observation:

The Pandas expression correctly replicates SQL semantics: row filtering with a condition and selecting only required columns. Retrieved records match the expected subset exactly.

Task 4: Real-Time Application: Multi-Language Snippet

Code:

```python
def task4_multilanguage_converter(work_dir: Path) -> TaskResult:
    print("\n# Task 4 - Real-Time Multi-Language Snippet Converter")

    scenario = "Translate Python business logic (grade classification) to JavaScript for web use."
    python_snippet = textwrap.dedent(
        """
        def grade(score: int) -> str:
            if score >= 90:
                return "A"
            if score >= 75:
                return "B"
            if score >= 60:
                return "C"
            return "D"
        """
    ).strip()

    js_snippet = textwrap.dedent(
        """
        function grade(score) {
          if (score >= 90) return "A";
          if (score >= 75) return "B";
          if (score >= 60) return "C";
          return "D";
        }

        const inputs = [95, 82, 67, 40];
        for (const s of inputs) {
          console.log(`${s}:${grade(s)}`);
        }
        """
    ).strip()

    print("Scenario:", scenario)
    print("\nPython snippet:")
    print(python_snippet)
    print("\nTranslated JavaScript snippet:")
    print(js_snippet)
```

Output:

```
Python output:
95:A
82:B
67:C
40:D

JavaScript output:
95:A
82:B
67:C
40:D
```

Observation:

Both Python and JavaScript implementations produce identical outputs for the same inputs. Main translation differences were syntax, string formatting style, and language runtime/tooling, not business logic.